

Paielement en ligne avec Stripe

Implémentation complète avec **React** (front) + **Node.js/Express** (back)

Ce document te donne une structure de projet, des exemples de code et une check-list de production : sessions Checkout, Payment Element, webhooks, tests et bonnes pratiques de sécurité.

Pré-requis : Node.js 18+, npm/pnpm, un compte Stripe, Git.

Référence : documentation Stripe (docs.stripe.com) et Stripe CLI.

Sommaire

1. Architecture & principes
2. Démarrage rapide (structure projet)
3. Option A - Stripe Checkout (recommandé pour commencer)
4. Option B - Payment Element (paiement intégré)
5. Webhooks (indispensable en production)
6. Tests, cartes de test & Stripe CLI
7. Mise en production : check-list

Astuce : commence par **Stripe Checkout** (Option A), puis passe au **Payment Element** si tu veux une UI 100% intégrée.

1. Architecture & principes

Un paiement Stripe se fait presque toujours en **2 parties** :

- **Front (React)** : affiche le bouton/formulaire, collecte les infos, déclenche la confirmation.
- **Back (Node.js)** : crée la transaction chez Stripe, stocke les **clés secrètes**, reçoit les **webhooks**.

Règles d'or : (1) jamais de clé secrète côté front, (2) ne livre jamais un produit uniquement parce que l'UI affiche "succès" - valide via webhook.

Choisir entre Checkout et Payment Element

Checkout : page de paiement hébergée par Stripe, rapide et robuste.

Payment Element : paiement dans ta page, plus de contrôle UI, mais plus d'intégration.

2. Démarrage rapide (structure projet)

Exemple de monorepo simple : un dossier **server** (Express) et un dossier **client** (React).

```
mon-projet/  
  server/  
    server.js  
    .env  
  client/  
    src/  
    .env  
  README.md
```

Variables d'environnement

```
# server/.env  
STRIPE_SECRET_KEY=sk_test_...  
STRIPE_WEBHOOK_SECRET=whsec_... # si webhooks  
FRONT_URL=http://localhost:5173
```

```
# client/.env  
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_...
```

Installs (exemple)

```
# server  
cd server  
npm init -y  
npm i express stripe cors dotenv  
  
# client (Vite)  
cd ../client  
npm create vite@latest . -- --template react  
npm i  
npm i @stripe/stripe-js @stripe/react-stripe-js
```

3. Option A - Stripe Checkout (recommandé pour commencer)

Flux : le client clique sur "Payer", ton serveur crée une **Checkout Session**, renvoie une URL, puis React redirige vers Stripe.

3.1. Serveur Express : créer une Checkout Session

```
// server/server.js (extrait)
app.post("/api/create-checkout-session", async (req, res) => {
  const { items } = req.body; // [{ name, priceInCents, quantity }]

  const session = await stripe.checkout.sessions.create({
    mode: "payment",
    line_items: items.map((i) => ({
      price_data: {
        currency: "eur",
        product_data: { name: i.name },
        unit_amount: i.priceInCents,
      },
      quantity: i.quantity,
    })),
    success_url: `${process.env.FRONT_URL}/success?session_id={CHECKOUT_SESSION_ID}`,
    cancel_url: `${process.env.FRONT_URL}/cancel`,
  });

  res.json({ url: session.url });
});
```

3.2. React : rediriger vers Checkout

```
// client/src/PayButton.jsx (extrait)
export function PayButton() {
  const pay = async () => {
    const r = await fetch("http://localhost:4242/api/create-checkout-session", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        items: [{ name: "Produit demo", priceInCents: 600, quantity: 1 }],
      }),
    });

    const { url, error } = await r.json();
    if (url) window.location.href = url;
    else alert(error ?? "Erreur paiement");
  };

  return <button onClick={pay}>Payer 6€</button>;
}
```

Important : ne valide pas la commande sur /success. Utilise un webhook pour confirmer le paiement (section 5).

4. Option B - Payment Element (paiement intégré)

Flux : ton serveur crée un **PaymentIntent**. Ton client affiche le **Payment Element** puis appelle **confirmPayment**.

4.1. Serveur : créer un PaymentIntent

```
// server/server.js (extrait)
app.post("/api/create-payment-intent", async (req, res) => {
  const { amountInCents } = req.body;

  const pi = await stripe.paymentIntents.create({
    amount: amountInCents,
    currency: "eur",
    automatic_payment_methods: { enabled: true },
  });

  res.json({ clientSecret: pi.client_secret });
});
```

4.2. React : le formulaire (CheckoutForm)

```
// client/src/CheckoutForm.jsx
import { PaymentElement, useElements, useStripe } from "@stripe/react-stripe-js";
import { useState } from "react";

export function CheckoutForm() {
  const stripe = useStripe();
  const elements = useElements();
  const [loading, setLoading] = useState(false);

  const onSubmit = async (e) => {
    e.preventDefault();
    if (!stripe || !elements) return;

    setLoading(true);
    const { error } = await stripe.confirmPayment({
      elements,
      confirmParams: { return_url: `${window.location.origin}/success` },
    });
    setLoading(false);

    if (error) alert(error.message);
  };

  return (
    <form onSubmit={onSubmit}>
      <PaymentElement />
      <button disabled={!stripe || loading}>{loading ? "...": "Payer"}</button>
    </form>
  );
}
```

4.3. React : page qui récupère le clientSecret (PaymentPage)

```
// client/src/PaymentPage.jsx (extrait)
import { useEffect, useState } from "react";
import { loadStripe } from "@stripe/stripe-js";
import { Elements } from "@stripe/react-stripe-js";
import { CheckoutForm } from "../CheckoutForm";

const stripePromise = loadStripe(import.meta.env.VITE_STRIPE_PUBLISHABLE_KEY);

export default function PaymentPage() {
  const [clientSecret, setClientSecret] = useState("");

  useEffect(() => {
    fetch("http://localhost:4242/api/create-payment-intent", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ amountInCents: 600 }),
    })
      .then((r) => r.json())
      .then((d) => setClientSecret(d.clientSecret));
  }, []);

  if (!clientSecret) return null;

  return (
    <Elements stripe={stripePromise} options={{ clientSecret }}>
      <CheckoutForm />
    </Elements>
  );
}
```

Conseil : calcule le montant côté serveur à partir de ton catalogue/panier (ne fais pas confiance au client).

5. Webhooks (indispensable en production)

Un webhook est une notification envoyée par Stripe à ton serveur (ex : "paiement réussi"). C'est la source de vérité pour marquer une commande comme payée et livrer/activer ton produit.

5.1. Endpoint webhook Express

Le webhook Stripe nécessite le corps de requête brut (raw body). Place ce route **avant** `express.json()` ou utilise un middleware dédié pour ce chemin.

```
// server/server.js (extrait)
app.post("/webhook", express.raw({ type: "application/json" })), (req, res) => {
  const sig = req.headers["stripe-signature"];

  let event;
  try {
    event = stripe.webhooks.constructEvent(req.body, sig, process.env.STRIPE_WEBHOOK_SECRET);
  } catch (err) {
    return res.status(400).send(`Webhook Error: ${err.message}`);
  }

  if (event.type === "checkout.session.completed") {
    const session = event.data.object;
    // TODO: marquer la commande payée (DB)
  }

  if (event.type === "payment_intent.succeeded") {
    const pi = event.data.object;
    // TODO: marquer la commande payée (DB)
  }

  res.json({ received: true });
});
```

Pour relier Stripe à ta base, stocke un identifiant métier (`commande_id`) via **metadata** ou un mapping DB.

6. Tests, cartes de test & Stripe CLI

Stripe fournit une CLI pour tester les webhooks en local.

6.1. Stripe CLI (écoute et redirige les webhooks vers localhost)

```
stripe login
```

```
stripe listen --forward-to localhost:4242/webhook  
# La CLI affiche un secret whsec_... à mettre dans server/.env
```

6.2. Cartes de test

Utilise la doc Stripe pour la liste la plus à jour. Exemple courant :

```
4242 4242 4242 4242  
Date future (ex: 12/34) - CVC (ex: 123)
```

7. Mise en production : check-list

- Utiliser HTTPS partout (front + back).
- Mettre les clés Stripe dans des variables d'environnement (jamais dans Git).
- Vérifier la signature des webhooks et gérer l'idempotence (éviter les doubles traitements).
- Calculer le montant côté serveur à partir de ton catalogue/panier (ne pas faire confiance au client).
- Gérer le cycle de vie : réussi, en attente, échoué, remboursement, litige.
- Journaliser les erreurs et garder des traces (id PaymentIntent / Session).
- Tester en mode test, puis activer la prod (pk_live / sk_live) et refaire un test complet.

Pour aller plus loin : abonnements (Billing), paiement récurrent, factures, TVA - à adapter selon ton modèle.

Fin du document.